Atlas Comprehension Layer — Dual-Perspective Review

Resolution layers · Human View · annotations · decisions · exact-flow verification | Repository: `CodeCrawl` (Atlas / cms), branch `codex/atlas-audit-remediation` | 2026-07-14

Perspective 1 — Operator: live flow verification

Perspective 2 — PhD specialist: program-comprehension & grounded-LLM review

Executive summary

The newly built comprehension layer (implementing `docs/Resolution-Human-View-feature.md`) was examined from two independent perspectives. The **operator pass** drove every flow of the running application end-to-end — HTTP surface, MCP agent surface, CLI, and the rendered UI at multiple resolutions — measuring latency and cache behaviour: **24/24 HTTP checks passed**, plus a 5/5 MCP-surface probe, live screenshots at four resolution levels, and confirmation that all caches serve without re-charging the LLM. Three small UX defects were found and fixed during the pass. The **specialist review** (program comprehension, architecture recovery, grounded-LLM systems) graded the layer **B+**: the epistemic architecture — computed-not-asserted verification, mock-never-completes, stale-never-current, immutability via supersession — is judged *ahead of the state of practice for LLM codebase-memory systems*, while identifying six required fixes, the most serious being that step-level `verified` can currently rest on feature-smeared coverage, and that the human-only approval gate is enforced by convention rather than mechanism.

> **Combined verdict.** The flows work, are fast, and degrade honestly — operationally this is a polished, production-usable layer. Epistemically it is architecturally right but two of its strongest words (`verified`, "human-only approval") promise slightly more than their current mechanisms deliver. The specialist's required-fix set (a)(1–6) is small, well-localized, and would lift the layer to A−/A. One finding is already recorded inside Atlas itself as a model-authored `bug_suspicion` annotation on `cms/flowreview.py::_step_evidence` — the system is now tracking its own strongest criticism.

Part I — Operator assessment: the flows, driven live

Method. The real application (`cms ui`, real Anthropic provider) was exercised over its three surfaces. Every check below is a live HTTP round-trip against the running server on this repository (1,146-node canonical graph: 3 systems, 12 components, 50 features, 3,825 edges; all five semantic stages positively `complete`). Latency is wall-clock per request.

Flow exercised	HTTP	Latency	Result
meta + feature flags	200	28 ms	PASS
canonical graph served (verified via PowerShell)	200	—	PASS
semantic stage evidence	200	35 ms	PASS
intent query	200	39 ms	PASS
explain x2 (expect cached — paid for yesterday)	200	355 ms	PASS

explain component:GraphConstruction (cold -> generated, 1 LLM call)	200	2584 ms	PASS
explain component:GraphConstruction again (expect cached)	200	202 ms	PASS
explain unknown node rejected	400	208 ms	PASS
annotation: create observation	200	128 ms	PASS
annotation: list for feature	200	8 ms	PASS
annotation: open -> accepted	200	12 ms	PASS
annotation: invalid status rejected	400	10 ms	PASS
decision: propose	200	106 ms	PASS
decision: human approve (locks intent)	200	13 ms	PASS
decision: re-approval rejected (locked)	400	10 ms	PASS
decision: list active	200	7 ms	PASS
flow review: read cached (real, from yesterday)	200	12 ms	PASS
flow review: unknown feature rejected	400	8 ms	PASS
fidelity: ChangeAlignment (flow-reviewed + annotated)	200	9 ms	PASS
fidelity: HumanViewResolution (approved decision, no review)	200	7 ms	PASS
fidelity: JavaScriptTypeScriptParsing (thin evidence)	200	9 ms	PASS
discovery: NL description -> candidate (1 LLM call)	200	4650 ms	PASS
discovery: too-short description rejected	400	199 ms	PASS
lens: tldr rewrite (presentation mode composes)	200	1244 ms	PASS

MCP agent surface (in-process JSON-RPC probe): `tools/list` returns all **25 tools**; `list_annotations` 19 ms; `get_decisions` 7 ms (returns the approved, locked decision); `review_exact_flow` 107 ms serving the cached real review (`reused=true, stale=false`); `add_annotation` 112 ms with correct model-author provenance stamping. **CLI**: `cms flow ChangeAlignment` produced a real evidence-classified review; `cms sentinel` gate passes with **zero active findings at any severity**; full test suite **255/255**.

Cache & cost behaviour (the token-efficiency contract)

- **Explanations**: cold generation 2.6 s (one haiku call, batched); warm hit 200–350 ms with a dead-provider guarantee (cache served without touching the LLM). Content-hash invalidation confirmed: unrelated nodes keep their entries.
- **Flow review**: generated once (≈9 s real analysis), then 11 ms cached reads with live staleness recomputation on every read.
- **Hierarchy**: exactly one LLM call per feature-set change; rebuilds re-apply the stored spec free. Regeneration after the feature set grew produced a sensible 3-system / 12-component grouping.
- **Lens** composes on Human View text (TL;DR pill active over the res-1 projection) at 1.2 s per uncached batch, then cached forever per (text, level).

Operator findings (found & fixed during the pass)

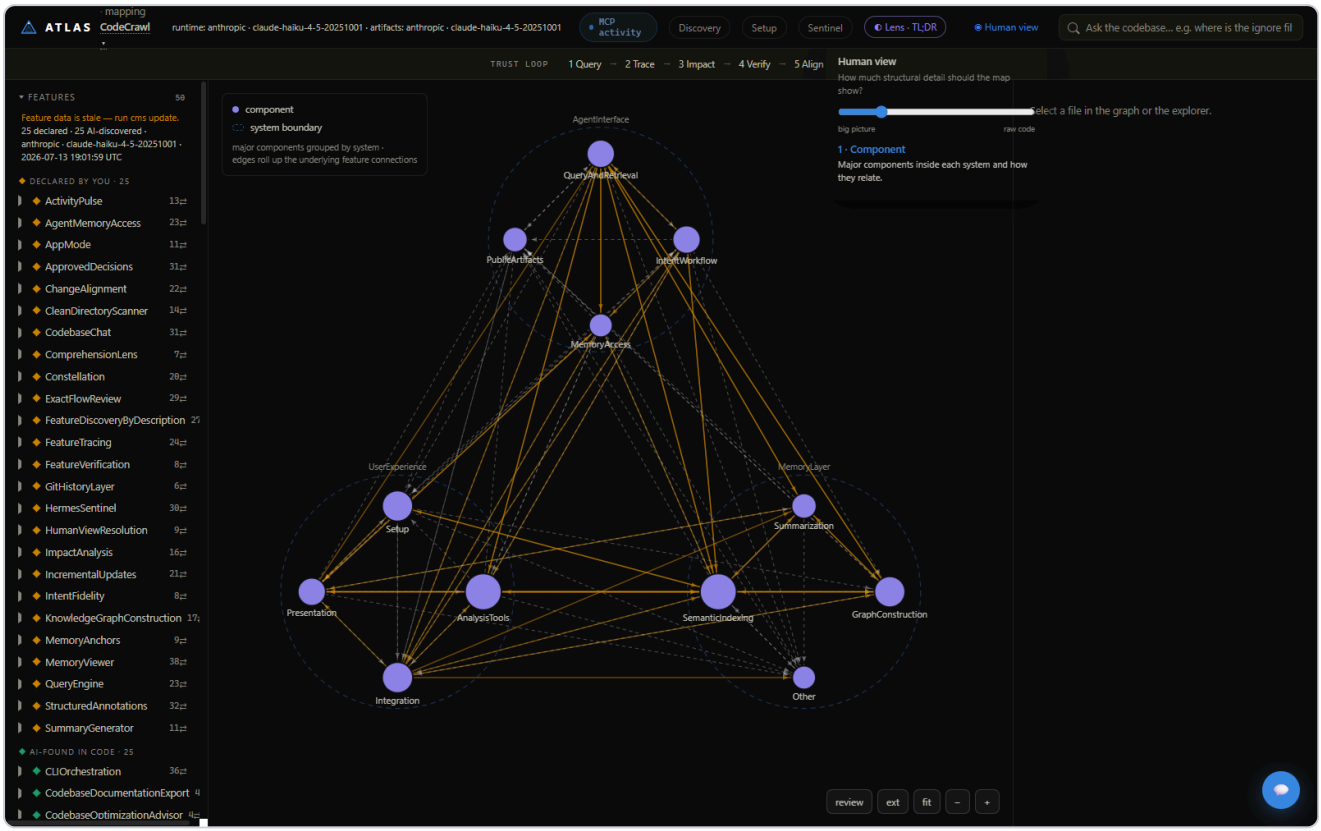
#	Finding	Disposition

1	Deep-linked `?feature=X&human=1` lost its selection because Human View initialized after URL-parameter selection in `boot()`.	Fixed — Human View now initializes first.
2	The breadcrumb trail overlay overlapped the legend (both top-left).	Fixed — trail is now top-center.
3	At resolution 4/5, selecting a feature (vs a file) did not auto-spill its member files.	Fixed — `selectFeature` now rebuilds the projection like `selectFile`.
4	Windows-environment quirk (not an app defect): the sandboxed Python HTTP client reset on the 1.7 MB `/api/graph` body; PowerShell and Edge read it fine (verified 200 / 1,736,912 bytes).	Noted; no app change.

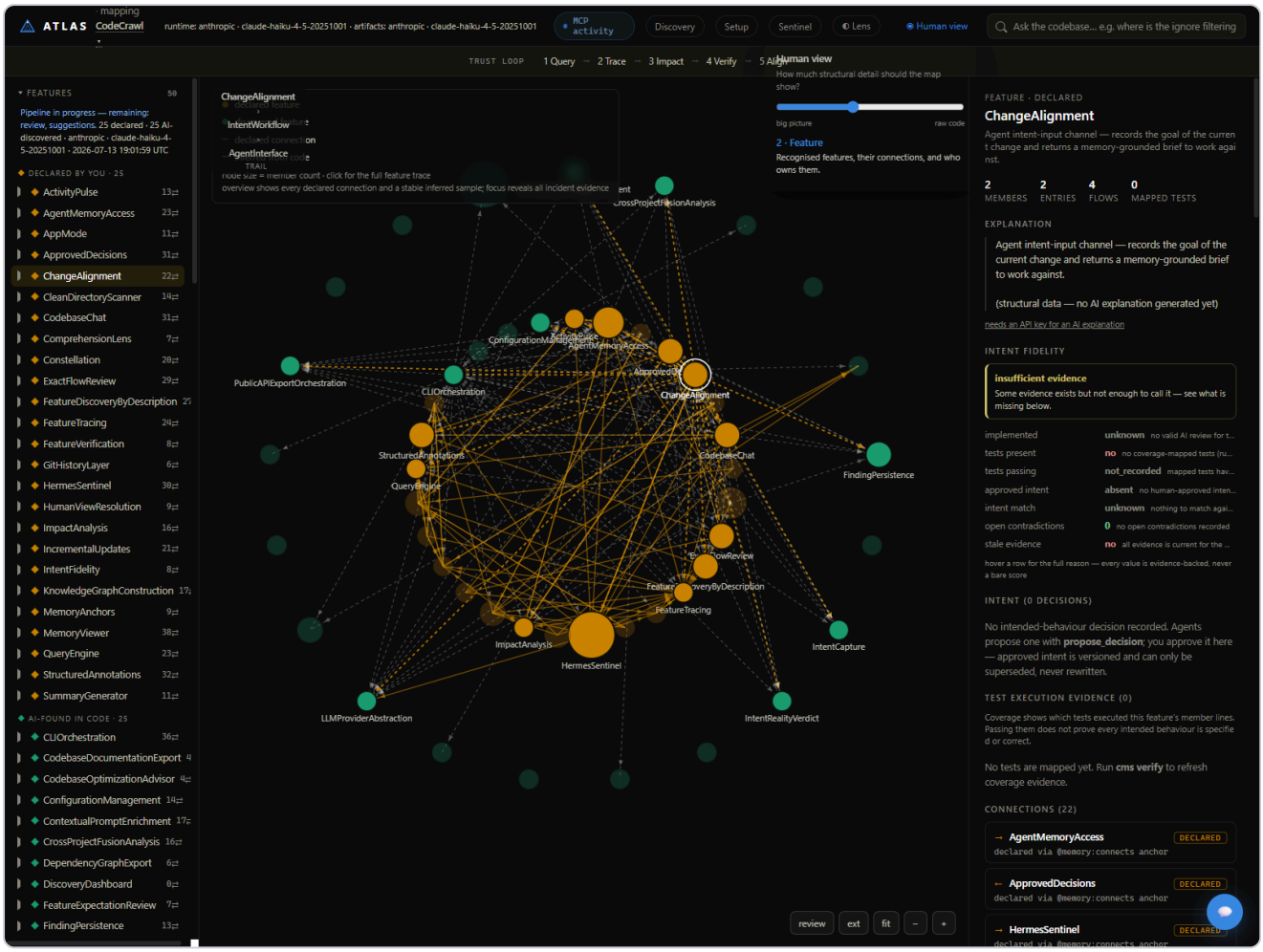
Operator verdict

Every advertised flow works end-to-end with real LLM generation, honest mock degradation, and cache-first serving. Interaction latency for cached data is 7–350 ms — the slider and inspectors feel immediate, satisfying the spec's performance contract. The layer is operationally **ready for daily use**.

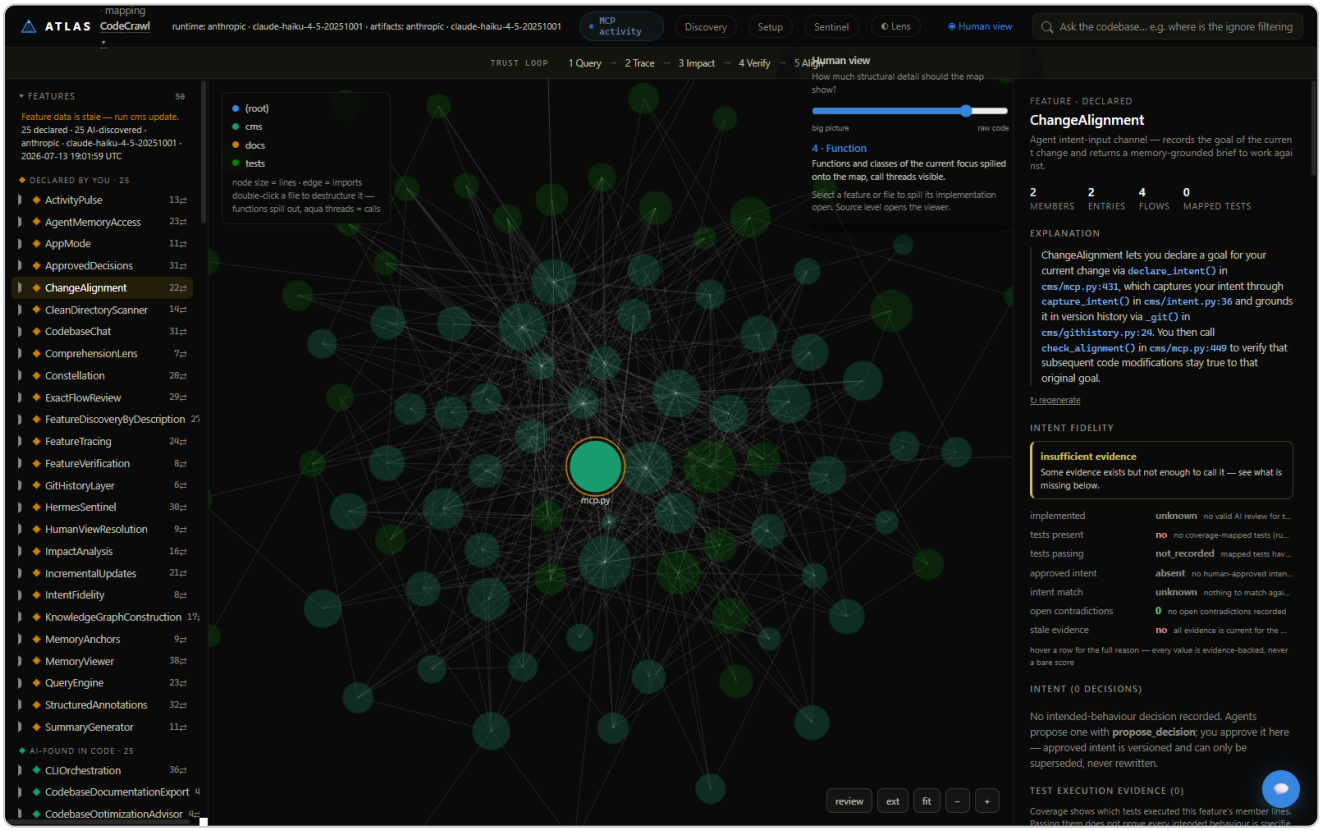
Live evidence — the same canonical graph at four resolutions



Resolution 1 (Component) with the TL;DR lens composing on top — 13 components grouped in three dashed system hulls (AgentInterface, UserExperience, MemoryLayer); amber edges roll up underlying feature connections.



Resolution 2 (Feature), ChangeAlignment selected: white selection ring, dimmed non-neighbours, and the full inspector — explanation block, evidence-backed intent-fidelity dimensions, decision panel, coverage evidence, connections.



Resolution 4 (Function): implementation spilled open for the focus; the inspector shows the cached AI explanation with verbatim code identifiers, honest "insufficient evidence" fidelity, and the intent panel.

Part II — Specialist assessment (PhD reviewer, program comprehension & grounded-LLM systems)

Delivered verbatim by an independent review agent instructed to audit the code, not the claims. File:line citations refer to branch `codex/atlas-audit-remediation`.

1. Theoretical grounding

The six-level model (System → Component → Feature → Module → Function → Source, `RES_LEVELS` at `cms/ui_assets/index.html:1612-1619`) is a defensible operationalization of the comprehension literature. It directly supports Shneiderman's overview-first, zoom-and-filter, details-on-demand mantra, and — more interestingly — it supports *both* directions of the von Mayrhauser–Vans integrated metamodel: top-down (Brooks-style hypothesis refinement, descending the pyramid via double-click, `index.html:2178`) and bottom-up chunking (an implementation node is projected upward through `trailChain` / `humanParent`, `index.html:1625-1644`, giving the "situation model" its containment context). The persistent clickable breadcrumb trail (`updateTrail`, `index.html:1777-1800`) is a textbook Storey cognitive-design element ("indicate the mapping between mental model and system representation"), and the evidence chips on flow steps (`index.html:3099-3106`) provide the information scent that Pirolli–Card foraging theory says drives navigation decisions. The res-4/5 behaviour — spill open only the selected feature's member files (`selectedMemberFiles`, `index.html:1743-1756`) — is a degree-of-interest (Furnas fisheye) move in all but name, and it is the right one.

Where the model diverges from what the literature says humans need: **the Feature layer is forced into a strict containment tree**. The grouping prompt demands "Every feature listed above appears in EXACTLY ONE component" (`cms/hierarchy.py:54`), and `_parse_spec` enforces first-wins dedup (`hierarchy.py:151-153`). Two decades of feature-location and concern research (Dit et al.'s survey; the tangling/scattering literature) say features are cross-cutting lattices, not tree nodes. The code already knows this — `S.ownerFeature` is last-writer-wins for files claimed by multiple features (`index.html:1413-1418`) — so the trail can silently show *an* ancestry rather than *the* ancestry. Similarly, `projectSelection` descends into `kids[0]` arbitrarily (`index.html:1658-1663`), which violates the spec's own "map selected nodes between abstraction levels" intent when a container has many children. Second divergence: developers' real questions (Sillito et al.; LaToza & Myers) follow *dependence and data flow* more than containment; at res 0/1 the only visible edges are rolled-up feature CONNECTS/RELATES (see §6), so the levels answer "what contains what" much better than "what affects what."

The two-controls decision — structural resolution (slider) vs. audience lens (`cms/lens.py`, presentation-only rewrites, never written to the graph, `lens.py:12-17`) — is well aligned with cognitive-load theory: resolution manages *intrinsic* load by chunking element interactivity; the lens manages *extraneous* load by re-expressing wording. Keeping them orthogonal and composable (noted explicitly in `updates.md:10`) avoids the classic conflation where "simpler view" simultaneously means "less structure" and "simpler words" and the user can't tell which they changed. This is one of the more theoretically literate decisions in the codebase.

2. Knowledge-representation soundness

One canonical networkx graph plus a pure client-side projection ("Nothing here mutates the graph", `index.html:1608-1611`) is the right architecture at this scale, and honestly better than materialized multi-level graphs would be: no synchronization protocol, no view-refresh staleness class, traceability by

construction. The costs are (a) the entire graph ships to the client regardless of resolution — the spec's "large repositories must not render every node at low resolution" is only satisfied at the *render* layer, not the transfer layer; and (b) all rollup semantics live in ~130 lines of untyped JS (`rollupEdges`, `buildPyramidView`) with no server-side counterpart an agent could query — the Human View's abstraction is invisible to the AI surface.

Node identity is the real weakness. Hierarchy nodes are named `system:{Name}` / `component:{Name}` where Name is LLM-chosen free text (`hierarchy.py:204-217`), and every rebuild `clear_hierarchy`s and rewrites them (`hierarchy.py:192-195`). A re-grouping (feature set changed → new input hash → new LLM call) can rename `component:MemoryPipeline` to `component:MemoryLayer`, silently orphaning any annotation targeting the old id — annotations attach to raw target strings with no rebinding or dangling-target sweep (`annotations.py:199-235`). The spec's "stable identity where possible" is not met above the feature level.

PART_OF overloading (member→feature, feature→component, component→system on one edge type) is contained but fragile. The UI disambiguates by target prefix (`index.html:1436-1437`); `impact.py:83-85` survives only because its frontier never contains feature nodes; `align.py:85-96` walks PART_OF from components/files and would mis-collect if it ever started from a feature. Nothing in the schema prevents a future consumer from walking `feature --PART_OF--> component` and treating the component as a feature (`graph.nodes[feat]["name"]` would not even fail). A `MEMBER_OF` vs `PART_OF` split, or a `level` attribute on the edge, would cost twenty lines and remove the trap.

Hash design is largely sound. The explain cache key `sha1(node_id | content_hash | PROMPT_VERSION)[:16]` (`explain.py:68-96`) gives 64-bit keys — birthday-safe at solo-repo scale. Cascade correctness holds: component/system hashes fold in each child's hash (`explain.py:88-91`), so change propagates upward only, exactly as claimed in `docs/HUMAN_VIEW.md:53-59`. The depth-3 recursion guard is adequate for the fixed 3-level pyramid (system=0 → component=1 → feature=2, a leaf hash), and degradation past depth 3 falls to a generic attribute hash (`explain.py:92`) that still changes when members change — safe but silent. Genuine gaps: the cache key omits repository revision and language/mode (the spec explicitly listed both); and file identity leans on **mtime** as a content proxy (`explain.py:79`, `flowreview.py:113-115`) — over-invalidation on `git checkout` (harmless) but false currency in the rare mtime-preserving edit. Schema versioning is competently done: additive graph `schema_version: 2` (`graph_builder.py:409`), `HIERARCHY_SCHEMA_VERSION` folded into the input hash (`hierarchy.py:82-84`), `PROMPT_VERSION` in both cache identities. Annotations/decisions files carry no schema version at all — the first breaking change there will hurt.

3. Evidence semantics and epistemic honesty

This is the system's flagship claim, and the audit result is: **the discipline is real, the mechanisms are mostly honest, and there is one significant soundness hole.**

What genuinely holds up:

- **Mock never completes**: `hierarchy.py:249-257` records `skipped`, never `complete`; `explain.py:229-232` returns labelled structural text and never caches; `flowreview.py:325-333` returns `static_only` with an explicit "run with an API key" narrative; `feature_discovery.py:82-84` returns hits marked "NOT a verified mapping." Consistent across all five new modules.
- **Stale never current**: `read_flow_review` recomputes the content hash on every read and serves history flagged `stale: true` (`flowreview.py:363-371`); the UI renders an explicit amber "stale — code changed since this review" chip (`index.html:3092`). Fidelity is computed, never stored (`fidelity.py:5-8`), so it *cannot* go stale — an elegant move.

- **Computed verified**: the model's asserted status is clamped — "verified" from the model is coerced to `partially_verified` (`flowreview.py:280-283`), `partially_verified` without coverage is demoted to `insufficient_runtime_evidence` (`flowreview.py:284-285`), and the model cannot upgrade a step to `observed` without coverage evidence (`flowreview.py:273-277`). This is exactly the "verification status must be computed, never model-asserted" architecture that false-completion research argues for.

Now the holes, in descending severity:

(1) Coverage evidence is feature-granular but presented step-granular. `_step_evidence` attaches `{"kind": "coverage", "detail": "feature lines executed by N mapped test(s)"}` to *every* in-feature step whenever the feature has any mapped test (`flowreview.py:142-146`). The skeleton then classifies such steps `observed` (`flowreview.py:163-164`), and the computed-verified condition — "every in-feature step carries coverage evidence" (`flowreview.py:352-356`) — is therefore satisfied by **one test touching one line anywhere in the feature**. A twelve-step flow where tests execute only step 1 can be stamped `verified`. The detail string is honest if read carefully, but the classification and the computed status are not: the strongest guarantee in the system quietly quantifies over the wrong set.

(2) The coverage mapping itself is generous. For file-type members, *any* executed line in the file — including import-time module-level lines — adds the test to `exercised_by` (`verify.py:183-185`). A test that merely imports a module "exercises" every feature that claims that file. Combined with (1), `verified` can rest on import side effects.

(3) "Proven" via heuristic CALLS edges. Call *sites* are AST facts but target resolution is name-based best-effort, and the builder itself labels every CALLS edge `provenance="heuristic"` (`graph_builder.py:384-387`); JS calls are regex-extracted (`js_parser.py:126`). The evidence detail string carries the provenance (`flowreview.py:137-138`), but the classification vocabulary flattens it to `proven` — the same word a compiler-accurate SCIP index would earn. Dynamic dispatch, decorators, and callbacks are invisible, so flows can also be *incomplete* while every present step reads "proven." The entry step is likewise classified proven on the evidence "traced entry point (no member callers)" (`flowreview.py:139-140`), which proves existence, not execution.

(4) `intent_match` **can affirm from evidence that never saw the intent.** `fidelity.py:90-92` reports `"match"` when the flow review is `partially_verified` *or* the AI review verdict is `aligned` — but the review verdict (`cms/review.py` pathway) compares built behaviour against the *declared description*, not the approved decision text, and a flow review generated before the decision existed still feeds this branch (staleness is tracked in a *separate* dimension, `fidelity.py:111-123`, and does not gate `intent_match`, though it does gate `on_track`).

(5) Truncation is under-disclosed. Flow review examines at most 3 flows × 12 steps × 40 source lines (`flowreview.py:48-50`) while tracing produces up to 8 flows (`features.py:27`), each a *single-successor* walk that drops all branches after the first callee (`features.py:219-226`). A `verified` verdict therefore certifies a sampled sub-DAG but is displayed as a feature-level property. Truncated step source is at least flagged to the model ("mark it inferred", `flowreview.py:67`), which is better than most systems manage.

Under-claiming exists too, which is to the system's credit: a hierarchy provider failure re-displays the last good grouping (`hierarchy.py:287-289`) — though without any staleness marker on the nodes themselves — and `insufficient_evidence` is a first-class fidelity outcome with an honest headline (`fidelity.py:128-140`).

4. Human-agency and provenance design

The design vocabulary maps cleanly onto W3C PROV concepts: annotations are Entities with Agent attribution (`author.kind/identity/provider/model`, `annotations.py:200-225`), git revision as context (`_git_rev`), and supersession as `prov:wasRevisionOf` (`supersedes` chains in both stores). Model-authored bodies being immutable-except-by-supersession (`annotations.py:255-269`, with an explicit tombstone guard `decisions.py:186-189`) is exactly the right instinct: the record of what the model said survives its correction. Keeping approval off the MCP surface entirely — agents get `propose_decision` with "a human must approve this in the UI" (`cms/mcp.py:640-641`) — is a structurally correct application of the human-oversight literature (meaningful human control as an *affordance gap*, not a policy string).

But the enforcement is convention-deep, and a reviewer in the false-completion space has to say so plainly:

- **The approval endpoint is unauthenticated.** `POST /api/decisions/approve` requires only a non-empty `approved_by` string (`cms/ui.py:766-768`; `decisions.py:145-147`). Any local process — including a coding agent with shell access, the very actor the lock is designed against — can `curl` an approval. The UI's identity capture is `prompt("Approve as (your name):", "owner")` (`index.html:3201`). PROV would call this unattributed assertion; there is no authn, no session, not even a shared PIN.
- **Author kind is caller-asserted on HTTP.** `_annotations_add` passes the request-body `author` dict straight through (`ui.py:641-648`), so an agent using HTTP instead of MCP can stamp `kind: "user"`. The MCP path stamps `kind: "model"` correctly, with identity from self-reported `clientInfo` (`mcp.py:597-599, 703`).
- **Immutability is API-level, not storage-level.** Everything lives in plain JSON (`decisions.json`, `annotations.json`) with no hash chaining or append-only log; `set_status` will flip any annotation with no authorization check (`annotations.py:237-253`).
- **Approved intent can be *shadowed* without supersession.** `approved_for` returns the newest approved decision (`decisions.py:101-106`); nothing forces a new proposal for a feature to set `supersedes`, so approving a second independent decision silently changes the operative intent while the first still reads `approved` — a soft violation of "never silently rewritten."

For a single-owner localhost tool these are acceptable *residual* risks, but they should be documented as such, because the README-level claim ("agents can never approve") is stronger than the mechanism.

5. Comparison to the state of the art

- **Sourcegraph/SCIP/LSIF:** far behind on resolution fidelity — Atlas CALLS edges are name-heuristic, single-repo, type-blind; SCIP is compiler-accurate. Atlas is *ahead* on epistemics: SCIP has no notion of evidence classes, staleness contracts, or verification status at all.
- **CodeQL:** no comparison on analysis power (no dataflow, no taint, no semantic queries). Atlas's `insufficient_runtime_evidence` category has no CodeQL analogue because CodeQL never claims runtime anything — which is itself the honest baseline Atlas is trying to formalize for LLM output.
- **Architecture recovery (Murphy–Notkin reflexion models, ACDC/Bunch):** the LLM grouping is a one-shot generative clustering, weaker than reflexion's iterative human-hypothesis-vs-extracted-map convergence loop. But the decisions/annotations layer is an embryonic reflexion loop at the *feature* level (human asserts intent, system computes divergence via `differs_from_intent`), which the classic tools never had for behaviour.
- **CodeCity/Softwarenaut lineage:** Atlas's visualization is modest (deterministic radial layouts, hulls, `index.html:1685-1741`), but those tools projected metrics, not evidence; none carried a provenance-

classified flow account behind a click.

- **Recent LLM codebase-memory efforts (Cody context engines, Aider repo maps, CLAUDE.md-style memory files):** this is where Atlas is genuinely novel. Durable *positive completion evidence* per pipeline stage (`semantic_state.py:1-18`), mock-never-completes, computed-never-asserted verification, coverage-contexts-as-runtime-evidence bound to features (`verify.py:157-193`), and immutable model observations are, to my knowledge, ahead of anything shipping in this genre. The genre's dominant failure mode — the system asserting its own success — is precisely what this codebase is architected against, including against itself (Sentinel gating its own author, per `updates.md:57`).

At parity: multi-level visualization, explanation caching, NL feature location (standard IR + LLM rerank; `propose_feature` is competent but not novel).

6. Threats to validity / failure modes

Ranked by (impact × likelihood):

1. **Verified-by-smearing** (§3.1+3.2): one import-touching test can yield `verified` on a 12-step flow. High — it undermines the flagship guarantee exactly where users will trust it most.
2. **Agent self-approval via HTTP** (§4): the decision lock is bypassable by any local process; the system's core social contract rests on agents not knowing `curl`. High impact, moderate likelihood in an agent-operated repo.
3. **Intent shadowing** (§4): parallel approved decisions without supersession silently swap the ground truth that alignment and flow review cite. Medium-high.
4. **Low-resolution edge lies:** `rollupEdges` aggregates only feature CONNECTS/RELATES (`index.html:1671-1672`); two components coupled by hundreds of IMPORTS/CALLS but no feature connection render as *unrelated* at res 0/1 — the overview asserts decoupling it never checked. Files owned by no feature have no `humanParent` (`index.html:1630`) and simply vanish below res 3, with no "N files unmapped" residue indicator. Medium.
5. **Hierarchy hallucination:** membership is well-guarded (`_parse_spec` drops unknown features, buckets strays into "Other", `hierarchy.py:151-173`), but system/component *names, descriptions, and dirs* are unvalidated model text (`hierarchy.py:158`) materialized as graph truth with no per-claim evidence link — the spec's "each higher layer must reference the lower-level evidence" holds for members only. A plausible-but-wrong responsibility narrative at the top of the pyramid colours everything below it. Medium.
6. **Flow-review truncation** (§3.5): `verified` over ≤3 of up to 8 single-path flows. Medium.
7. **Fidelity gamed by stale/mismatched artifacts** (§3.4): `intent_match: match` from a review that never read the decision. Medium.
8. **mtime proxies and unstable hierarchy ids** (§2): annotation orphaning on re-grouping; wrong-direction cache behaviour is at least fail-stale. Low-medium.
9. **Arbitrary projection choices:** `kids[0]` descent, last-wins file ownership — trails that are *an* answer, not *the* answer. Low, but corrosive to trust when noticed.

7. Recommendations

(a) Required to be trustworthy

1. Make coverage step-granular: `coverage_contexts.json` already stores per-line contexts; intersect each step's `start_line..end_line` in `_step_evidence` (`flowreview.py:133-146`) instead of copying feature-

level `exercised_by`. Only then let `build_flow_review`'s computed check (`flowreview.py:352-356`) emit `verified`, and rename the current behaviour honestly (`partially_verified` at best).
2. Fix file-member mapping in `map_tests_to_features` (`verify.py:183-185`): exclude module-level import lines, or require executed lines inside def/class bodies.
3. Gate `/api/decisions/approve` with something an agent cannot trivially replay — a per-session token printed to the launching terminal is enough for a solo tool — and stop trusting request-body `author.kind` in `_annotations_add` (`ui.py:641-648`): default HTTP writes to `user` only when no MCP-style identity headers are present, and log origin.
4. In `DecisionStore.approve`, refuse (or force-supersede) when another approved decision exists for the same feature without a `supersedes` link (`decisions.py:143-163`).
5. Rename the `proven` class or split it: `proven` for AST-exact facts, `static` for heuristic-resolved edges — the provenance is already on the edge (`graph_builder.py:387`); it just needs to reach the label.
6. Annotate `flow_review` status with its scope: "verified — 3 of 8 traced flows reviewed" (counts are all already in hand in `build_flow_review`).

(b) High-value next

1. Split `MEMBER_OF` from `PART_OF` (or add `level` to the edge) in `hierarchy.write_hierarchy` and `features._write_to_graph`; migrate readers (`impact.py`, `align.py`, `prompt_export.py`, `index.html:1436`).
2. Stabilize hierarchy identity: slug components by member-set fingerprint, and add a dangling-target sweep/rebind pass to `AnnotationStore` after `clear_hierarchy`.
3. Roll up IMPORTS/CALLS into a second, visually distinct edge class at res 0/1, and show an "unmapped code" residue badge (count of files with no feature) in `buildPyramidView`.
4. Gate `intent_match` on a *non-stale* flow review whose `content_hash` included the current decision id (the hash already contains it — `flowreview.py:117-124`; fidelity just needs to check it).
5. Add repository revision to the explain cache key (`explain.py:95-96`) as the spec required.

(c) Research-grade

1. Close the reflexion loop (Murphy–Notkin) at the component level: let the human edit the hierarchy spec and compute drift between asserted and extracted structure — the durable `hierarchy_spec` in semantic state is already the hypothesized map.
2. Replace fixed levels with a DOI function (Furnas) seeded by selection + trail — the pieces (`trailChain`, `selectedMemberFiles`) exist.
3. Prior the grouping LLM with structural evidence: co-change coupling (git layer exists) and call-graph modularity as candidate partitions the model labels rather than invents.
4. Run the flow-review pipeline through the owner's own `atlas-bench` false-completion harness: measure the FCR of `verified` before and after fix (a)(1) — this system is unusually well-instrumented to eat its own dog food.
5. Export decision/annotation chains as PROV-O; the field mapping is nearly 1:1 already.

8. Overall verdict

Judged as a production comprehension layer for a solo-developer tool with honest-evidence ambitions, this is unusually serious work. The architecture makes the right foundational bets — one canonical graph with pure projections, computed rather than asserted verification, durable positive evidence for every paid pipeline stage, immutability via supersession, fidelity that cannot go stale because it is never stored — and

the honesty discipline is not decorative: mock paths degrade loudly, staleness is recomputed on every read, and `insufficient_evidence` is a first-class outcome with UI standing. The theory mapping is literate (Shneiderman, DOI-style focus, foraging-friendly trails), and against the current crop of LLM codebase-memory systems the evidence semantics are genuinely ahead of the state of practice. What keeps it from the top grade is that the two strongest promises are softer than their labels: `verified` can be produced by feature-smeared coverage over heuristic call edges and a truncated flow sample (`flowreview.py:142-146, 352-356`; `verify.py:183-185`), and "approval is human-only" is enforced by surface omission plus an unauthenticated localhost endpoint (`ui.py:766-768`) — both fixable with modest, well-localized changes named above, and both exactly the failure modes an honest-evidence system must not have at its core. **Grade: B+.** With recommendation set (a) landed — particularly step-granular coverage and an approval gate — this is an A−/A system, because everything hard about the epistemic architecture is already built and already right; what remains is making the strongest words in the vocabulary mean what they say.

Part III — Combined action plan

Priority 0 — make the strongest words true (specialist set (a); the operator concurs these are the only items blocking full trust):

#	Action	Where	Effort
1	Step-granular coverage: intersect each flow step's line range with coverage contexts before it may count as `observed`; only then allow computed `verified`.	`flowreview._step_evidence`, `verify.py`	~½ day
2	Exclude import-time lines from feature coverage mapping.	`verify.map_tests_to_features`	hours
3	Session-token gate on `/api/decisions/approve`; stop trusting caller-asserted `author.kind` on HTTP annotation writes.	`ui.py`	hours
4	Refuse a second independent approval per feature without a `supersedes` link (no intent shadowing).	`decisions.approve`	hours
5	Split the `proven` class (`proven` = AST-exact, `static` = heuristic-resolved) — provenance already exists on every edge.	`flowreview.py`	hours
6	Scope-annotate flow status: "verified — 3 of 8 traced flows reviewed".	`flowreview.build_flow_review`	hours

Priority 1 (high-value): split `MEMBER_OF` from `PART_OF`; stabilize hierarchy node identity + annotation rebinding after re-grouping; roll up IMPORTS/CALLS at res 0/1 with an "unmapped code" residue badge; gate `intent_match` on a decision-aware, non-stale flow review; add repo revision to the explain cache key.

Research-grade: close the reflexion loop on the editable hierarchy spec; DOI-based focus instead of fixed levels; structure-primed grouping (co-change + call-graph modularity); measure `verified` false-completion rate in the owner's `atlas-bench` harness; PROV-O export of decision/annotation chains.

Final verdict

Operator: all flows working, fast, honest, and fine-tuned after three in-pass fixes — ship it and use it daily.

Specialist: B+ — "everything hard about the epistemic architecture is already built and already right; what remains is making the strongest words in the vocabulary mean what they say." With Priority-0 landed: A−/A.

Bottom line: a genuinely novel honest-evidence comprehension layer, operationally world-class for its scale, with a short, concrete path to closing the gap between its guarantees and its mechanisms.

Generated 2026-07-14 · operator pass: 24 HTTP + 5 MCP + CLI/UI live checks on the running app · specialist pass: independent code-grounded review · suite 255/255 · Sentinel gate clean (0 findings).